

Simulación de Concurrency para el SIGET: Productor-Consumidor con Semáforos

Relatoría técnica

Autor: Alexandra Zapata Moreno

Asignatura: Sistemas Operativos

Fecha: 11/09/2026

1. Introducción y objetivo

El objetivo de este trabajo es implementar una solución de software que demuestre un control y una sincronización de concurrencia efectivos en un escenario simulado del SIGET (Sistema Inteligente de Gestión de Tráfico). Para ello se eligió el problema clásico del productor-consumidor, adaptado al contexto del tráfico urbano. En esta adaptación, varios sensores de tráfico actúan como productores que envían sus lecturas a un búfer compartido, mientras que varios módulos de análisis actúan como consumidores que procesan esas lecturas. El propósito específico es garantizar que no haya pérdida ni corrupción de información, previniendo condiciones de carrera y evitando interbloqueos mediante el uso de semáforos y mutex.

2. Lógica de la implementación

La solución se implementó en el lenguaje Python. Se utilizó la librería `threading` para crear hilos concurrentes y las primitivas `threading.Semaphore` y `threading.Lock` para la sincronización.

Se definieron tres componentes principales. El primero es la clase `BufferCompartido`, que representa un búfer de capacidad fija con dos punteros (entrada y salida) y un arreglo circular. Esta clase utiliza tres mecanismos de sincronización: un semáforo llamado `vacios`, que se inicializa con la capacidad del búfer y cuenta cuántas posiciones libres hay; un semáforo llamado `lentos`, que se inicializa en cero y cuenta cuántos datos están disponibles para consumir; y un mutex que protege el acceso al arreglo interno para evitar condiciones de carrera.

El segundo componente es la clase `Sensor`, que hereda de `Thread` y simula un sensor de tráfico. Cada sensor genera lecturas con información de zona, tipo de medición, valor y hora, y las deposita en el búfer. El tercer componente es la clase `Analizador`, que también hereda de `Thread` y simula un módulo de análisis que extrae datos del búfer y los procesa. Las estadísticas compartidas y la lista de eventos se actualizan bajo un candado adicional para garantizar la consistencia del monitoreo.

El flujo de sincronización es el siguiente. Cuando un productor desea escribir un dato, primero ejecuta `vacios.acquire()`. Si el búfer está lleno, este semáforo lo bloquea hasta que haya espacio disponible. Cuando logra continuar, adquiere el mutex, escribe el dato en el arreglo, libera el mutex y ejecuta `llenos.release()`, lo que indica que hay un dato nuevo disponible. El consumidor realiza la operación inversa: ejecuta `llenos.acquire()` y, si el búfer está vacío, queda bloqueado hasta que llegue un dato; luego adquiere el mutex, extrae el dato, libera el mutex y ejecuta `vacios.release()`, lo que indica que hay una posición libre más.

Este orden estricto de operaciones es el que previene tanto las condiciones de carrera como los interbloqueos. Además, el consumidor utiliza `acquire()` con un tiempo de espera máximo de 0.3 segundos. Esto le permite revisar periódicamente si el programa debe finalizar, evitando que los hilos queden bloqueados indefinidamente cuando ya no hay más datos por procesar.

3. Observaciones durante la ejecución

Durante la ejecución se observó un comportamiento concurrente estable y equilibrado. Los tres sensores producían datos de manera independiente mientras los dos analizadores los consumían en paralelo, sin que el búfer se desbordara ni se quedara sin elementos cuando había trabajo pendiente. El tablero en tiempo real permitió comprobar que los semáforos vacíos y llenos siempre sumaban la capacidad total del búfer, lo que confirma la consistencia del estado compartido.

También se pudo observar con claridad que, cuando el búfer se llenaba, el semáforo vacíos bajaba a cero y los productores se bloqueaban de forma automática. De la misma manera, cuando el búfer se vaciaba, el semáforo llenos bajaba a cero y los consumidores se bloqueaban. Al liberarse espacio o al ingresar nuevos datos, los hilos se despertaban automáticamente en el orden correcto.

Al finalizar la simulación, la verificación de integridad mostró que el número de datos producidos era exactamente igual al número de datos consumidos, sin pérdidas ni duplicaciones. En el caso ejecutado fueron 18 datos producidos y 18 datos consumidos, con cero pérdidas.

4. Prevención de problemas de concurrencia

Condiciones de carrera. Se previenen con el mutex que protege el arreglo interno del búfer. Toda escritura o lectura ocurre dentro de una sección crítica, y ninguna operación sobre el arreglo puede intercalarse con otra.

Interbloqueos (deadlocks). Se previenen porque los semáforos se adquieren siempre en el mismo orden: primero el semáforo del recurso (vacíos o llenos) y después el mutex. Nunca

un hilo adquiere el mutex antes que un semáforo. Además, cada `acquire()` tiene su correspondiente `release()`, garantizado mediante bloques `try` y `finally`.

Inanición. Se evita porque los semáforos de Python reparten los permisos de manera justa; ningún hilo queda esperando indefinidamente si otro hilo libera el recurso de manera repetida.

Desbordamiento y vaciado indebido del búfer. Se evitan porque el semáforo vacío bloquea a los productores cuando el búfer está lleno, y el semáforo lleno bloquea a los consumidores cuando el búfer está vacío.

5. Reflexión y conclusiones

Esta simulación demuestra que la concurrencia no consiste únicamente en ejecutar varias tareas al mismo tiempo, sino en coordinarlas correctamente para que el resultado sea íntegro y confiable. En el contexto del SIGET, donde un semáforo de emergencia no puede quedarse esperando a que un sensor de velocidad termine de escribir, y donde un módulo de análisis no puede leer datos que se están escribiendo a medias, la sincronización es un requisito crítico.

El uso combinado de semáforos contadores y mutex es el tipo de mecanismo que permite a un sistema crítico operar con fiabilidad, escalabilidad y sin colapsos. Los semáforos controlan el flujo de recursos y bloquean a los hilos solo cuando es necesario, mientras que el mutex garantiza la exclusión mutua en la sección crítica.

Como aprendizaje personal, comprendí la diferencia entre un semáforo y un mutex: el mutex protege un recurso exclusivo, mientras que el semáforo controla el acceso a un número limitado de recursos. También comprendí la importancia de adquirir los recursos siempre en el mismo orden para evitar interbloqueos, y de usar bloques `try` y `finally` para asegurar que los recursos se liberen aunque ocurra un error.

La concurrencia mejora la capacidad de respuesta y el rendimiento del sistema, pero introduce complejidad. Si no se gestiona bien, puede provocar corrupción de datos, interbloqueos e inanición. Este trabajo confirma que los mecanismos clásicos de sincronización estudiados en clase son suficientes y necesarios para construir software concurrente confiable, y que su aplicación correcta es lo que distingue a un sistema robusto de uno que colapsa bajo carga.